Part 1: Import Pandas

Open **Jupyter Notebook** and create a new notebook.

1. Load the CSV

Our file is:

Rural_Water_Supply_Project_Analytics_500.csv

If the CSV is in the same folder as your Jupyter Notebook:

```
import pandas as pd
df = pd.read_csv("Google Drive/Rural_Water_Supply_Project_Analytics_500.csv")
```

---

2. Look at the data

```
df.head()
```

	Project_ID	State	District	Block	Village	Contractor	Start_Date	Planned_End_Date	Actual_End_Date	Pipeline_Length_KM	Planned
0	RWS-0001	Karnataka	Bengaluru Rural	Doddaballapur	Doddaballapur Village 1	Krishna Civil Projects	2024-04-12	2025-04-06	NaN	33.5	
1	RWS-0002	Karnataka	Bengaluru Rural	Devanahalli	Devanahalli Village 1	Bharat Waterworks	2025-07-14	2026-08-16	NaN	23.0	
2	RWS-0003	Karnataka	Bengaluru Rural	Nelamangala	Nelamangala Village 1	Bharat Waterworks	2025-04-29	2025-12-15	2025-12-16	25.1	
3	RWS-0004	Karnataka	Bengaluru Rural	Hoskote	Hoskote Village 1	Nirmal Pipeline Works	2024-07-24	2025-04-10	NaN	12.0	
4	RWS-0005	Karnataka	Mysuru	Hunsur	Hunsur Village 1	Nirmal Pipeline Works	2024-12-03	2025-09-19	NaN	29.7	

This shows the **first 5 rows**.

You can see more rows:

Last 5 rows:

df.tail()

---

3. Find number of rows and columns

```
df.shape
```

```
(500, 20)
```

This means:

500 rows × 20 columns

---

4. See column names

```
df.columns
```

```
Index(['Project_ID', 'State', 'District', 'Block', 'Village', 'Contractor',
       'Start_Date', 'Planned_End_Date', 'Actual_End_Date',
       'Pipeline_Length_KM', 'Planned_Cost_INR', 'Actual_Cost_INR',
       'Planned_Progress_Pct', 'Actual_Progress_Pct', 'Delay_Days',
       'Total_Manpower', 'Skilled_Workers', 'Unskilled_Workers', 'Supervisors',
       'Project_Status'],
      dtype='object')
```

---

5. Check the data types

```
df.dtypes
```

```
Project_ID              object
State                   object
District                object
Block                   object
Village                 object
Contractor              object
Start_Date              object
Planned_End_Date        object
Actual_End_Date         object
Pipeline_Length_KM     float64
Planned_Cost_INR         int64
Actual_Cost_INR        float64
Planned_Progress_Pct   float64
Actual_Progress_Pct    float64
Delay_Days               int64
Total_Manpower           int64
Skilled_Workers          int64
Unskilled_Workers        int64
Supervisors              int64
Project_Status          object
dtype: object
```

1) **What does object mean?**
   Usually it means **text/string data**.
   For example:
   Karnataka
   Mysuru
   Hassan
   Completed
2) **What does int64 mean?**
   Whole numbers:
   100
   250
   500
3) **What does float64 mean?**
   Numbers that can contain decimals:
   12.5
   35.7
   82.3

---

6. Get a quick summary

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 20 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Project_ID            500 non-null    object
 1   State                 500 non-null    object
 2   District              500 non-null    object
 3   Block                 500 non-null    object
 4   Village               500 non-null    object
 5   Contractor            500 non-null    object
 6   Start_Date            500 non-null    object
 7   Planned_End_Date      500 non-null    object
 8   Actual_End_Date       80 non-null     object
 9   Pipeline_Length_KM    499 non-null    float64
 10  Planned_Cost_INR      500 non-null    int64
 11  Actual_Cost_INR       499 non-null    float64
 12  Planned_Progress_Pct  500 non-null    float64
 13  Actual_Progress_Pct   500 non-null    float64
 14  Delay_Days            500 non-null    int64
 15  Total_Manpower        500 non-null    int64
 16  Skilled_Workers       500 non-null    int64
 17  Unskilled_Workers     500 non-null    int64
 18  Supervisors           500 non-null    int64
 19  Project_Status        500 non-null    object
dtypes: float64(4), int64(6), object(10)
memory usage: 78.3+ KB
```

---

7. Check missing values

```
df.isnull().sum()
```

```
Project_ID              0
State                   0
District                0
Block                   0
Village                 0
Contractor              0
Start_Date              0
Planned_End_Date        0
Actual_End_Date       420
Pipeline_Length_KM      1
Planned_Cost_INR        0
Actual_Cost_INR         1
Planned_Progress_Pct    0
Actual_Progress_Pct     0
Delay_Days              0
Total_Manpower          0
Skilled_Workers         0
Unskilled_Workers       0
Supervisors             0
Project_Status          0
dtype: int64
```

---

8. Find the actual missing rows

```
df[df["Actual_Cost_INR"].isnull()]
```

Block	Village	Contractor	Start_Date	Planned_End_Date	Actual_End_Date	Pipeline_Length_KM	Planned_Cost_INR	Actual_Cost_INR	Planned_Progress_Pct	Actual_Prog
Kunigal	Kunigal Village 1	AquaBuild Infra Pvt Ltd	2025-03-13	2025-09-16	NaN	23.6	35347403	NaN	90.0	

```
df[df["Actual_End_Date"].isnull()]
```

	Project_ID	State	District	Block	Village	Contractor	Start_Date	Planned_End_Date	Actual_End_Date	Pipeline_Length_KM	Planned_Cost
0	RWS-0001	Karnataka	Bengaluru Rural	Doddaballapur	Doddaballapur Village 1	Krishna Civil Projects	2024-04-12	2025-04-06	NaN	33.5	4805
1	RWS-0002	Karnataka	Bengaluru Rural	Devanahalli	Devanahalli Village 1	Bharat Waterworks	2025-07-14	2026-08-16	NaN	23.0	2338
3	RWS-0004	Karnataka	Bengaluru Rural	Hoskote	Hoskote Village 1	Nirmal Pipeline Works	2024-07-24	2025-04-10	NaN	12.0	1740
4	RWS-0005	Karnataka	Mysuru	Hunsur	Hunsur Village 1	Nirmal Pipeline Works	2024-12-03	2025-09-19	NaN	29.7	3422
5	RWS-0006	Karnataka	Mysuru	Nanjangud	Nanjangud Village 1	Bharat Waterworks	2025-04-16	2025-12-14	NaN	29.3	4217
...	...	...	...	...	...	...	...	...	...	...	
495	RWS-0496	Maharashtra	Pune	Junnar	Junnar Village 4	Krishna Civil Projects	2024-05-08	2024-12-22	NaN	33.9	3371
496	RWS-0497	Maharashtra	Nashik	Dindori	Dindori Village 4	RuralFlow Engineering	2025-10-14	2026-05-04	NaN	14.8	2350
497	RWS-0498	Maharashtra	Nashik	Igatpuri	Igatpuri Village 4	Sahyadri Water Infra	2024-01-29	2025-02-05	NaN	33.8	4797
498	RWS-0499	Maharashtra	Nashik	Niphad	Niphad Village 4	Krishna Civil Projects	2025-08-05	2026-05-14	NaN	30.9	3803
499	RWS-0500	Maharashtra	Nashik	Sinnar	Sinnar Village 4	GramJal Contractors	2024-06-01	2025-07-10	NaN	21.3	2679

420 rows × 20 columns

```
df[df["Pipeline_Length_KM"].isnull()]
```

	Project_ID	State	District	Block	Village	Contractor	Start_Date	Planned_End_Date	Actual_End_Date	Pipeline_Length_KM	Planned_Cost_INR	Actual_Cost_INR
241	RWS-0242	Andhra Pradesh	Kurnool	Alur	Alur Village 2	BlueLine Water Solutions	2024-06-01	2025-07-22	NaN	NaN	23482901	23647752.(

9. Check duplicate records

```
df.duplicated().sum()
```

```
np.int64(0)
```

0 Means No Duplicate Records

```
df["Project_ID"].duplicated().sum()
```

```
np.int64(0)
```

---

10. Check unique project statuses

```
df["Project_Status"].unique()
```

```
array(['Ongoing', 'Delayed', 'Completed', 'At Risk'], dtype=object)
```

---

11. Count projects by status

```
df["Project_Status"].value_counts()
```

```
Project_Status
Ongoing      196
Delayed      125
At Risk       99
Completed     80
Name: count, dtype: int64
```

---

12. Check contractor names

```
df["Contractor"].unique()
```

```
array(['Krishna Civil Projects', 'Bharat Waterworks',
       'Nirmal Pipeline Works', 'AquaBuild Infra Pvt Ltd',
       'Shakti Infra Projects', 'Sahyadri Water Infra',
       'JalSetu Projects', 'BlueLine Water Solutions',
       'RuralFlow Engineering', 'GramJal Contractors',
       'RuralFlow Engineering '], dtype=object)
```

---

13. Clean the contractor column

We can remove unnecessary spaces using:

df["Contractor"] = df["Contractor"].str.strip()

What does .strip() do?

It removes unnecessary spaces from the beginning and end.

For example:

" RuralFlow Engineering "

becomes:

"RuralFlow Engineering"

Now check again:

df["Contractor"].unique()

```
df[df["Pipeline_Length_KM"].isnull()]
```

	Project_ID	State	District	Block	Village	Contractor	Start_Date	Planned_End_Date	Actual_End_Date	Pipeline_Length_KM	Planned_Cost_INR	Actual_Cost_INR
241	RWS-0242	Andhra Pradesh	Kurnool	Alur	Alur Village 2	BlueLine Water Solutions	2024-06-01	2025-07-22	NaN	NaN	23482901	23647752.0

---

14. Clean missing Pipeline Length

We have one missing pipeline length.

```
df[df["Pipeline_Length_KM"].isnull()]
```

	Project_ID	State	District	Block	Village	Contractor	Start_Date	Planned_End_Date	Actual_End_Date	Pipeline_Length_KM	Plann
241	RWS-0242	Andhra Pradesh	Kurnool	Alur	Alur Village 2	BlueLine Water Solutions	2024-06-01	2025-07-22	NaN	NaN	

```
df["Pipeline_Length_KM"].median()
```

```
20.1
```

```
df["Pipeline_Length_KM"] = df["Pipeline_Length_KM"].fillna(
    df["Pipeline_Length_KM"].median()
)
```

```
df["Pipeline_Length_KM"].isnull().sum()
```

```
np.int64(0)
```

---

15. Clean missing Actual Cost

```
df[df["Actual_Cost_INR"].isnull()]
```

	Project_ID	State	District	Block	Village	Contractor	Start_Date	Planned_End_Date	Actual_End_Date	Pipeline_Length_KM	Planned_Cost_INR	Actual_Co
17	RWS-0018	Karnataka	Tumakuru	Kunigal	Kunigal Village 1	AquaBuild Infra Pvt Ltd	2025-03-13	2025-09-16	NaN	23.6	35347403	

```
df["Actual_Cost_INR"] = df["Actual_Cost_INR"].fillna(
    df["Planned_Cost_INR"]
)
```

Why?

If actual cost isn't available, we're assuming the project currently has actual expenditure equal to its planned cost **for this synthetic training dataset**.

In a real project, you should instead investigate the source system or business owner before making this assumption.

---

16. Convert dates properly

```
df["Start_Date"] = pd.to_datetime(df["Start_Date"])

df["Planned_End_Date"] = pd.to_datetime(df["Planned_End_Date"])

df["Actual_End_Date"] = pd.to_datetime(df["Actual_End_Date"])
```

```
df.dtypes
```

```
Project_ID                    object
State                         object
District                      object
Block                         object
Village                       object
Contractor                    object
Start_Date            datetime64[ns]
Planned_End_Date      datetime64[ns]
Actual_End_Date       datetime64[ns]
Pipeline_Length_KM          float64
Planned_Cost_INR              int64
Actual_Cost_INR             float64
Planned_Progress_Pct        float64
Actual_Progress_Pct         float64
Delay_Days                    int64
Total_Manpower                int64
Skilled_Workers               int64
Unskilled_Workers             int64
Supervisors                   int64
Project_Status               object
dtype: object
```

Now Python understands that these are dates.

---

17. Create useful analytics columns

This is where our project gets interesting.

1) **Cost Variance**

   Create:

   ```
   df["Cost_Variance_INR"] = (
       df["Actual_Cost_INR"] - df["Planned_Cost_INR"]
   )
   ```

   Example:

   Planned Cost = ₹50 lakh

   Actual Cost = ₹55 lakh

   Cost Variance = ₹5 lakh

---

2) **Cost Variance %**

   ```
   df["Cost_Variance_Pct"] = (
       df["Cost_Variance_INR"] /
       df["Planned_Cost_INR"] * 100
   )
   ```

---

3) **Progress Variance**

   ```
   df["Progress_Variance_Pct"] = (
       df["Actual_Progress_Pct"] -
       df["Planned_Progress_Pct"]
   )
   ```

   For example:

   Planned = 80%

   Actual = 70%

   Progress Variance = -10%

   Negative means the project is behind planned progress.

```
df["Cost_Variance_INR"] = (
    df["Actual_Cost_INR"] - df["Planned_Cost_INR"]
)
```

```
df["Cost_Variance_Pct"] = (
    df["Cost_Variance_INR"] /
    df["Planned_Cost_INR"] * 100
)
```

```
df["Progress_Variance_Pct"] = (
    df["Actual_Progress_Pct"] -
    df["Planned_Progress_Pct"]
)
```

---

18. Check the cleaned dataset

1. head()

```
df.head()
```

	Project_ID	State	District	Block	Village	Contractor	Start_Date	Planned_End_Date	Actual_End_Date	Pipeline_Length
0	RWS-0001	Karnataka	Bengaluru Rural	Doddaballapur	Doddaballapur Village 1	Krishna Civil Projects	2024-04-12	2025-04-06	2025-04-06	
1	RWS-0002	Karnataka	Bengaluru Rural	Devanahalli	Devanahalli Village 1	Bharat Waterworks	2025-07-14	2026-08-16	2026-08-16	
2	RWS-0003	Karnataka	Bengaluru Rural	Nelamangala	Nelamangala Village 1	Bharat Waterworks	2025-04-29	2025-12-15	2025-12-16	
3	RWS-0004	Karnataka	Bengaluru Rural	Hoskote	Hoskote Village 1	Nirmal Pipeline Works	2024-07-24	2025-04-10	2025-04-10	
4	RWS-0005	Karnataka	Mysuru	Hunsur	Hunsur Village 1	Nirmal Pipeline Works	2024-12-03	2025-09-19	2025-09-19	

5 rows × 24 columns

2. info()

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 24 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Project_ID            500 non-null    object
 1   State                 500 non-null    object
 2   District              500 non-null    object
 3   Block                 500 non-null    object
 4   Village               500 non-null    object
 5   Contractor            500 non-null    object
 6   Start_Date            500 non-null    datetime64[ns]
 7   Planned_End_Date      500 non-null    datetime64[ns]
 8   Actual_End_Date       500 non-null    datetime64[ns]
 9   Pipeline_Length_KM    500 non-null    float64
 10  Planned_Cost_INR      500 non-null    int64
 11  Actual_Cost_INR       500 non-null    float64
 12  Planned_Progress_Pct  500 non-null    float64
 13  Actual_Progress_Pct   500 non-null    float64
 14  Delay_Days            500 non-null    int64
 15  Total_Manpower        500 non-null    int64
 16  Skilled_Workers       500 non-null    int64
 17  Unskilled_Workers     500 non-null    int64
 18  Supervisors           500 non-null    int64
 19  Project_Status        500 non-null    object
 20  Cost_Variance_INR     500 non-null    float64
 21  Cost_Variance_Pct     500 non-null    float64
 22  Progress_Variance_Pct 500 non-null    float64
 23  Is_Delayed            500 non-null    bool
dtypes: bool(1), datetime64[ns](3), float64(7), int64(6), object(7)
memory usage: 90.5+ KB
```

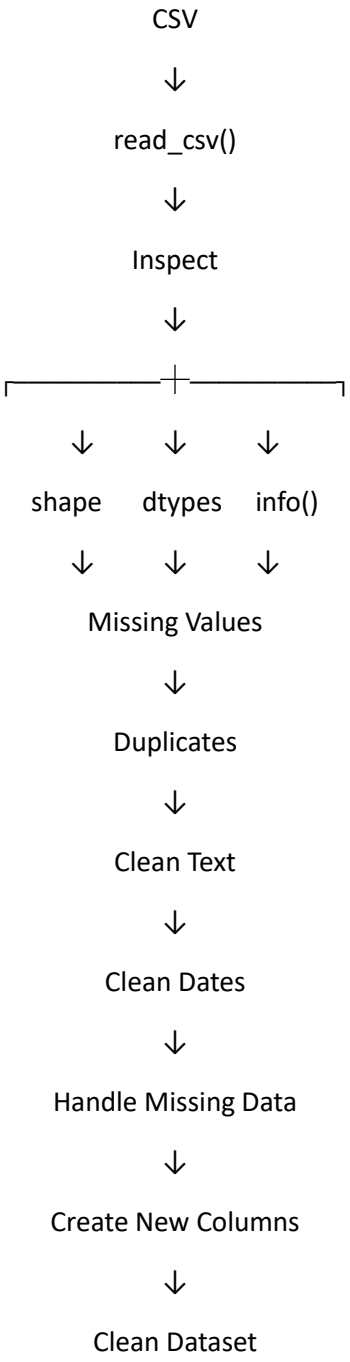
3. isnull().sum()

```
df.isnull().sum()
```

```
Project_ID              0
State                   0
District                0
Block                   0
Village                 0
Contractor              0
Start_Date              0
Planned_End_Date        0
Actual_End_Date         0
Pipeline_Length_KM      0
Planned_Cost_INR        0
Actual_Cost_INR         0
Planned_Progress_Pct    0
Actual_Progress_Pct     0
Delay_Days              0
Total_Manpower          0
Skilled_Workers         0
Unskilled_Workers       0
Supervisors             0
Project_Status          0
Cost_Variance_INR       0
Cost_Variance_Pct       0
Progress_Variance_Pct   0
Is_Delayed              0
dtype: int64
```

This Gives us the Idea that dataset has no Null value and it has been cleaned.

Data Analyst workflow so far



Part 2: EDA – Exploratory Data Analysis

- Which district has the most delayed projects?
- Which projects are over budget?
- What is the average pipeline length?
- Which contractor performs best?
- Is higher project cost associated with delays?

---

1. First, make sure your cleaned data is ready

- By Following the steps in **Part 1.**

---

2. Understand the overall project portfolio

df.shape()

```
df.shape
```

```
(500, 23)
```

df.describe()

describe() gives you statistics for numerical columns such as:

- Pipeline length
- Planned cost
- Actual cost
- Progress
- Delay
- Manpower

```
df.describe()
```

	Start_Date	Planned_End_Date	Actual_End_Date	Pipeline_Length_KM	Planned_Cost_INR	Actual_Cost_INR	Planned_Progress_Pct	Actual_Progress_Pct	De
count	500	500	80	500.000000	5.000000e+02	5.000000e+02	500.000000	500.000000	5(
mean	2024-12-21 17:48:28.799999744	2025-10-20 21:21:36.000000256	2025-10-20 12:54:00	19.892800	2.581000e+07	2.663016e+07	87.840000	84.386800	2
min	2024-01-01 00:00:00	2024-07-28 00:00:00	2024-07-28 00:00:00	4.000000	5.757662e+06	5.983751e+06	70.000000	41.500000	
25%	2024-06-30 18:00:00	2025-05-02 00:00:00	2025-05-12 12:00:00	12.475000	1.701328e+07	1.725889e+07	80.000000	76.375000	
50%	2024-12-26 00:00:00	2025-10-26 12:00:00	2025-12-07 00:00:00	20.100000	2.551051e+07	2.526952e+07	90.000000	84.700000	
75%	2025-06-05 00:00:00	2026-04-10 12:00:00	2026-03-17 12:00:00	27.600000	3.318290e+07	3.423027e+07	95.000000	93.900000	
max	2025-11-26 00:00:00	2027-01-18 00:00:00	2026-11-09 00:00:00	34.900000	5.603508e+07	6.382981e+07	100.000000	100.000000	
std	NaN	NaN	NaN	8.752566	1.129054e+07	1.214876e+07	8.454428	11.654168	2

Few More Problems:-

1) What is the average project cost?
   df["Planned_Cost_INR"].mean()
2) Average actual cost:

df["Actual_Cost_INR"].mean()

3) Average pipeline length:
df["Pipeline_Length_KM"].mean()

4) Average delay:
df["Delay_Days"].mean()

---

3. Project status analysis

How many projects are Completed, Ongoing, At Risk and Delayed?

For Counts:

```
df["Project_Status"].value_counts()
```

```
Project_Status
Ongoing      196
Delayed      125
At Risk       99
Completed     80
Name: count, dtype: int64
```

For percentages:

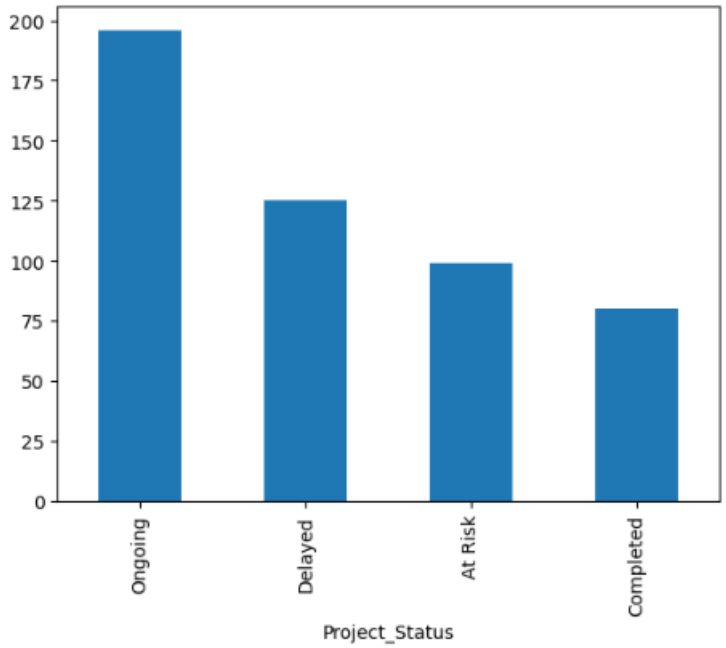
```
df["Project_Status"].value_counts(normalize=True) * 100
```

```
Project_Status
Ongoing      39.2
Delayed      25.0
At Risk      19.8
Completed    16.0
Name: proportion, dtype: float64
```

Making a simple chart:

```
df["Project_Status"].value_counts().plot(kind="bar")
```

```
<Axes: xlabel='Project_Status'>
```



What are we looking for?

For example, suppose the result tells us:

Completed 150

Ongoing 170

Delayed 110

At Risk 70

We could say:

"Out of 500 projects, 110 are delayed and 70 are currently at risk."

That's a **business insight**, not just a Python output.

---

4. Find all delayed projects

Now let's specifically investigate delayed projects.

```
delayed_projects = df[df["Project_Status"] == "Delayed"]
```

```
delayed_projects.head()
```

	Project_ID	State	District	Block	Village	Contractor	Start_Date	Planned_End_Date	Actual_End_Date	Pipeline_Length_KM	...	Delay_Days	Total_I
1	RWS-0002	Karnataka	Bengaluru Rural	Devanahalli	Devanahalli Village 1	Bharat Waterworks	2025-07-14	2026-08-16	2026-08-16	23.0	...	48	
3	RWS-0004	Karnataka	Bengaluru Rural	Hoskote	Hoskote Village 1	Nirmal Pipeline Works	2024-07-24	2025-04-10	2025-04-10	12.0	...	39	
7	RWS-0008	Karnataka	Mysuru	T Narasipura	T Narasipura Village 1	Shakti Infra Projects	2025-07-17	2026-04-18	2026-04-18	25.6	...	52	
9	RWS-0010	Karnataka	Mandya	Maddur	Maddur Village 1	Sahyadri Water Infra	2025-02-14	2025-12-27	2025-12-27	19.0	...	22	
13	RWS-0014	Karnataka	Hassan	Belur	Belur Village 1	JalSetu Projects	2025-10-01	2026-05-07	2026-05-07	33.0	...	55	

5 rows × 24 columns

```
len(delayed_projects)
```

```
125
```

---

5. Find the most delayed projects

Sort by delay:

```
df.sort_values(
    "Delay_Days",
    ascending=False
).head(10)
```

	Project_ID	State	District	Block	Village	Contractor	Start_Date	Planned_End_Date	Actual_End_Date	Pipeline_Length_KM	...	Actual_Pr
22	RWS-0023	Karnataka	Kolar	Srinivaspur	Srinivaspur Village 1	Sahyadri Water Infra	2025-03-19	2025-12-24	2025-12-24	20.2	...	
456	RWS-0457	Karnataka	Hassan	Arkalgud	Arkalgud Village 4	BlueLine Water Solutions	2025-09-08	2026-04-02	2026-04-02	32.9	...	
88	RWS-0089	Andhra Pradesh	Anantapur	Dharmavaram	Dharmavaram Village 1	AquaBuild Infra Pvt Ltd	2024-07-10	2025-01-22	2025-01-22	27.9	...	
384	RWS-0385	Andhra Pradesh	Anantapur	Dharmavaram	Dharmavaram Village 3	GramJal Contractors	2025-06-05	2026-07-02	2026-07-02	29.0	...	
30	RWS-0031	Karnataka	Shivamogga	Sorab	Sorab Village 1	Shakti Infra Projects	2024-06-25	2025-03-31	2025-03-31	4.3	...	
388	RWS-0389	Andhra Pradesh	Kurnool	Adoni	Adoni Village 3	Sahyadri Water Infra	2025-09-13	2026-06-07	2026-06-07	23.1	...	
357	RWS-0358	Maharashtra	Solapur	Barshi	Barshi Village 3	AquaBuild Infra Pvt Ltd	2025-07-03	2026-01-12	2026-01-12	34.5	...	
333	RWS-0334	Tamil Nadu	Dharmapuri	Palacode	Palacode Village 3	Krishna Civil Projects	2025-06-03	2026-03-31	2026-03-31	8.5	...	
161	RWS-0162	Karnataka	Hassan	Belur	Belur Village 2	Krishna Civil Projects	2024-09-16	2025-08-17	2025-08-17	31.3	...	
385	RWS-0386	Andhra Pradesh	Anantapur	Gooty	Gooty Village 3	Nirmal Pipeline Works	2024-05-08	2025-04-06	2025-04-06	26.9	...	

10 rows × 23 columns

We can make the output easier to read:

```
df[
    ["Project_ID", "District", "Village", "Contractor", "Delay_Days", "Project_Status"]
].sort_values(
    "Delay_Days",
    ascending=False
).head(10)
```

	Project_ID	District	Village	Contractor	Delay_Days	Project_Status
22	RWS-0023	Kolar	Srinivaspur Village 1	Sahyadri Water Infra	94	Delayed
456	RWS-0457	Hassan	Arkalgud Village 4	BlueLine Water Solutions	87	Delayed
88	RWS-0089	Anantapur	Dharmavaram Village 1	AquaBuild Infra Pvt Ltd	85	Delayed
384	RWS-0385	Anantapur	Dharmavaram Village 3	GramJal Contractors	83	Delayed
30	RWS-0031	Shivamogga	Sorab Village 1	Shakti Infra Projects	82	Delayed
388	RWS-0389	Kurnool	Adoni Village 3	Sahyadri Water Infra	80	Delayed
357	RWS-0358	Solapur	Barshi Village 3	AquaBuild Infra Pvt Ltd	78	Delayed
333	RWS-0334	Dharmapuri	Palacode Village 3	Krishna Civil Projects	77	Delayed
161	RWS-0162	Hassan	Belur Village 2	Krishna Civil Projects	76	Delayed
385	RWS-0386	Anantapur	Gooty Village 3	Nirmal Pipeline Works	76	Delayed

Business question answered:

Which projects require immediate management attention?

---

6. Find projects that are behind schedule

```
df.sort_values(
    "Progress_Variance_Pct"
).head(5)
```

	Project_ID	State	District	Block	Village	Contractor	Start_Date	Planned_End_Date	Actual_End_Date	Pipeline_Length_KM	...	Delay_Days
225	RWS-0226	Telangana	Nalgonda	Devarakonda	Devarakonda Village 2	Shakti Infra Projects	2025-02-22	2026-04-09	2026-04-09	28.7	...	36
250	RWS-0251	Odisha	Ganjam	Chhatrapur	Chhatrapur Village 2	JalSetu Projects	2024-11-18	2025-08-07	2025-08-07	6.2	...	0
475	RWS-0476	Karnataka	Shivamogga	Bhadravati	Bhadravati Village 4	GramJal Contractors	2025-04-15	2025-10-18	2025-10-18	24.6	...	35
298	RWS-0299	Karnataka	Bengaluru Rural	Nelamangala	Nelamangala Village 3	RuralFlow Engineering	2024-07-11	2025-08-06	2025-08-06	15.8	...	0
133	RWS-0134	Uttar Pradesh	Varanasi	Chiraigaon	Chiraigaon Village 1	AquaBuild Infra Pvt Ltd	2024-07-01	2025-04-23	2025-04-23	19.1	...	23

5 rows × 24 columns

These are our projects with the biggest progress shortfall.

---

7. Find over-budget projects

```
over_budget = df[df["Cost_Variance_INR"] > 0]
```

```
len(over_budget)
```

```
298
```

---

8. Calculate percentage of projects over budget

```
over_budget_percentage = (
    len(over_budget) / len(df)
) * 100

print(over_budget_percentage)
```

```
59.599999999999994
```

This answers:

What percentage of projects are exceeding their planned budget?

---

9. Find the biggest budget overruns

```
df[
    ["Project_ID", "District", "Contractor",
     "Planned_Cost_INR", "Actual_Cost_INR",
     "Cost_Variance_INR"]
].sort_values(
    "Cost_Variance_INR",
    ascending=False
).head()
```

	Project_ID	District	Contractor	Planned_Cost_INR	Actual_Cost_INR	Cost_Variance_INR
20	RWS-0021	Kolar	Bharat Waterworks	44378105	62807140.0	18429035.0
482	RWS-0483	Dharmapuri	Nirmal Pipeline Works	40119049	53811320.0	13692271.0
263	RWS-0264	Indore	BlueLine Water Solutions	38694454	51703324.0	13008870.0
225	RWS-0226	Nalgonda	Shakti Infra Projects	39845521	52689149.0	12843628.0
340	RWS-0341	Tiruchirappalli	GramJal Contractors	50927193	62986275.0	12059082.0

Now we can identify the projects causing the biggest financial overruns.

10. Analyze cost variance percentage

```
df[
    ["Project_ID", "Planned_Cost_INR",
     "Actual_Cost_INR", "Cost_Variance_Pct"]
].sort_values(
    "Cost_Variance_Pct",
    ascending=False
).head()
```

	Project_ID	Planned_Cost_INR	Actual_Cost_INR	Cost_Variance_Pct
20	RWS-0021	44378105	62807140.0	41.527314
482	RWS-0483	40119049	53811320.0	34.129102
263	RWS-0264	38694454	51703324.0	33.619469
225	RWS-0226	39845521	52689149.0	32.233555
312	RWS-0313	33215428	43739414.0	31.684029

This gives us the projects with the largest **percentage cost overruns**.

---

11. District-wise project summary

```
district_projects = df.groupby("District").size()

print(district_projects)
```

```
District
Ahmednagar         12
Ajmer              12
Alwar              12
Anantapur          12
Bengaluru Rural    16
Chitradurga        16
Chittoor           12
Dewas              12
Dharmapuri         16
Erode              16
Ganjam             12
Gaya               12
Hassan             16
Indore             12
Jaipur             12
Karimnagar         12
Khordha            12
Kolar              16
Kurnool            12
Lucknow            12
Mandya             16
Mysuru             16
Nalanda            12
Nalgonda           12
Nashik             16
Patna              12
Prayagraj          12
Pune               16
Salem              16
Sehore             12
Shivamogga         16
Solapur            12
Sundargarh         12
Tiruchirappalli    16
Tumakuru           16
Varanasi           12
Warangal           12
dtype: int64
```

12. District-wise average cost

```
district_cost = df.groupby("District")[
    "Actual_Cost_INR"
].mean()

print(district_cost)
```

```
District
Ahmednagar          2.435103e+07
Ajmer               2.217644e+07
Alwar               2.892116e+07
Anantapur           2.526410e+07
Bengaluru Rural     2.695193e+07
Chitradurga         2.308823e+07
Chittoor            3.682290e+07
Dewas               2.635193e+07
Dharmapuri          2.779334e+07
Erode               2.564074e+07
Ganjam              2.553597e+07
Gaya                2.778652e+07
Hassan              2.834044e+07
Indore              3.165985e+07
Jaipur              2.504347e+07
Karimnagar          2.136461e+07
Khordha             2.337829e+07
Kolar               2.755923e+07
Kurnool             2.840986e+07
Lucknow             3.097238e+07
Mandya              2.822164e+07
Mysuru              2.224501e+07
Nalanda             2.997983e+07
Nalgonda            2.794795e+07
Nashik              2.801947e+07
Patna               2.675387e+07
Prayagraj           2.562098e+07
Pune                2.445789e+07
Salem               3.054757e+07
Sehore              3.022349e+07
Shivamogga          2.431851e+07
Solapur             2.632712e+07
Sundargarh          2.505034e+07
Tiruchirappalli     2.491879e+07
Tumakuru            2.697602e+07
Varanasi            2.382435e+07
Warangal            2.371850e+07
Name: Actual_Cost_INR, dtype: float64
```

This tells us the average actual project cost in each district.

---

13. District-wise average delay

```
district_delay = df.groupby("District")[
    "Delay_Days"
].mean().sort_values(ascending=False)

print(district_delay)
```

```
District
Anantapur          38.000000
Hassan             37.312500
Varanasi           32.166667
Indore             31.583333
Nalgonda           31.333333
Erode              30.500000
Warangal           28.916667
Tiruchirappalli    26.750000
Kolar              26.312500
Dewas              26.166667
Sundargarh         25.333333
Mysuru             25.125000
Kurnool            24.416667
Ahmednagar         23.250000
Sehore             22.750000
Solapur            22.416667
Dharmapuri         22.312500
Bengaluru Rural    20.937500
Pune               20.812500
Nashik             20.437500
Alwar              19.750000
Salem              19.437500
Jaipur             19.416667
Khordha            19.333333
Ganjam             19.250000
Shivamogga         18.812500
Chittoor           18.666667
Karimnagar         18.416667
Chitradurga        18.000000
Tumakuru           15.812500
Gaya               15.000000
Mandya             14.625000
Prayagraj          14.166667
Nalanda            13.666667
Ajmer              13.666667
Lucknow            11.416667
Patna              11.083333
Name: Delay_Days, dtype: float64
```

Which districts have the highest average project delay?

Top 5:

```
district_delay.head(5)
```

```
District
Anantapur    38.000000
Hassan       37.312500
Varanasi     32.166667
Indore       31.583333
Nalgonda     31.333333
Name: Delay_Days, dtype: float64
```

---

14. District-wise progress

```
district_progress = df.groupby("District")[
    "Actual_Progress_Pct"
].mean().sort_values(ascending=False)

print(district_progress)
```

```
District
Karimnagar          92.050000
Prayagraj           90.933333
Sundargarh          89.725000
Ajmer               89.708333
Dewas               88.991667
Nashik              88.393750
Alwar               87.966667
Tumakuru            87.831250
Kolar               86.787500
Erode               86.743750
Mandya              86.256250
Ahmednagar          86.233333
Ganjam              85.608333
Nalanda             84.841667
Lucknow             84.833333
Hassan              84.437500
Chitradurga         84.350000
Mysuru              84.231250
Solapur             84.225000
Pune                83.781250
Salem               83.750000
Jaipur              83.658333
Warangal            83.450000
Dharmapuri          82.518750
Chittoor            82.133333
Patna               82.008333
Tiruchirappalli     82.000000
Varanasi            81.975000
Gaya                81.483333
Sehore              81.408333
Bengaluru Rural     81.375000
Indore              81.325000
Khordha             80.041667
Nalgonda            79.883333
Shivamogga          79.331250
Anantapur           79.233333
Kurnool             78.683333
Name: Actual_Progress_Pct, dtype: float64
```

This tells us which districts have the highest average actual progress.

---

15. Create a complete district summary

```
district_summary = df.groupby("District").agg(
    Total_Projects=("Project_ID", "count"),
    Average_Planned_Cost=("Planned_Cost_INR", "mean"),
    Average_Actual_Cost=("Actual_Cost_INR", "mean"),
    Average_Delay=("Delay_Days", "mean"),
    Average_Progress=("Actual_Progress_Pct", "mean"),
    Average_Pipeline_KM=("Pipeline_Length_KM", "mean")
)
```

```
district_summary
```

District	Total_Projects	Average_Planned_Cost	Average_Actual_Cost	Average_Delay	Average_Progress	Average_Pipeline_KM
Ahmednagar	12	2.343350e+07	2.435103e+07	23.250000	86.233333	17.616667
Ajmer	12	2.231427e+07	2.217644e+07	13.666667	89.708333	17.183333
Alwar	12	2.735110e+07	2.892116e+07	19.750000	87.966667	20.933333
Anantapur	12	2.513202e+07	2.526410e+07	38.000000	79.233333	19.308333
Bengaluru Rural	16	2.771127e+07	2.695193e+07	20.937500	81.375000	20.237500
Chitradurga	16	2.247634e+07	2.308823e+07	18.000000	84.350000	16.587500
Chittoor	12	3.465899e+07	3.682290e+07	18.666667	82.133333	25.566667
Dewas	12	2.572415e+07	2.635193e+07	26.166667	88.991667	20.491667
Dharmapuri	16	2.692980e+07	2.779334e+07	22.312500	82.518750	20.150000
Erode	16	2.461599e+07	2.564074e+07	30.500000	86.743750	18.712500
Ganjam	12	2.486811e+07	2.553597e+07	19.250000	85.608333	20.525000
Gaya	12	2.540426e+07	2.778652e+07	15.000000	81.483333	17.483333
Hassan	16	2.726488e+07	2.834044e+07	37.312500	84.437500	22.456250
Indore	12	2.827187e+07	3.165985e+07	31.583333	81.325000	22.216667
Jaipur	12	2.405202e+07	2.504347e+07	19.416667	83.658333	18.500000
Karimnagar	12	2.100629e+07	2.136461e+07	18.416667	92.050000	15.958333
Khordha	12	2.278907e+07	2.337829e+07	19.333333	80.041667	17.975000
Kolar	16	2.567855e+07	2.755923e+07	26.312500	86.787500	20.968750
Kurnool	12	2.732522e+07	2.840986e+07	24.416667	78.683333	22.841667
Lucknow	12	2.935027e+07	3.097238e+07	11.416667	84.833333	22.550000
Mandya	16	2.793627e+07	2.822164e+07	14.625000	86.256250	20.368750
Mysuru	16	2.223435e+07	2.224501e+07	25.125000	84.231250	18.581250
Nalanda	12	2.958815e+07	2.997983e+07	13.666667	84.841667	22.791667

We have now created a **district performance table**.

---

16. Count delayed projects by district

```
delayed_by_district = df[
    df["Project_Status"] == "Delayed"
].groupby("District").size().sort_values(
    ascending=False
)

print(delayed_by_district)
```

```
District
Anantapur          7
Nalgonda           6
Indore             6
Varanasi           6
Sehore             6
Bengaluru Rural    5
Hassan             5
Patna              4
Pune               4
Khordha            4
Shivamogga         4
Tiruchirappalli    4
Ahmednagar         4
Mysuru             4
Ganjam             4
Erode              4
Dharmapuri         4
Chittoor           4
Chitradurga        4
Tumakuru           3
Kurnool            3
Mandya             3
Kolar              3
Jaipur             3
Gaya               3
Warangal           3
Nalanda            2
Nashik             2
Salem              2
Solapur            2
Sundargarh         2
Dewas              2
Lucknow            1
Alwar              1
Karimnagar         1
dtype: int64
```

Now We can identify:

Which districts have the most delayed projects?

---

17. Contractor performance

```
df["Contractor"].value_counts()
```

```
Contractor
Krishna Civil Projects      56
Nirmal Pipeline Works       54
GramJal Contractors         53
AquaBuild Infra Pvt Ltd     52
RuralFlow Engineering       52
Bharat Waterworks           50
Sahyadri Water Infra        50
BlueLine Water Solutions    46
Shakti Infra Projects       45
JalSetu Projects            42
Name: count, dtype: int64
```

---

18. Projects handled by each contractor

```
contractor_projects = df.groupby("Contractor").size()

print(contractor_projects)
```

```
Contractor
AquaBuild Infra Pvt Ltd     52
Bharat Waterworks           50
BlueLine Water Solutions    46
GramJal Contractors         53
JalSetu Projects            42
Krishna Civil Projects      56
Nirmal Pipeline Works       54
RuralFlow Engineering       52
Sahyadri Water Infra        50
Shakti Infra Projects       45
dtype: int64
```

---

19. Average contractor delay

```
contractor_cost = df.groupby("Contractor")[
    "Delay_Days"
].mean().sort_values()

print(contractor_cost)
```

```
Contractor
GramJal Contractors         18.018868
AquaBuild Infra Pvt Ltd     19.903846
JalSetu Projects            20.642857
Krishna Civil Projects      20.642857
RuralFlow Engineering       21.173077
Sahyadri Water Infra        22.740000
Bharat Waterworks           23.340000
Shakti Infra Projects       23.888889
Nirmal Pipeline Works       25.185185
BlueLine Water Solutions    26.608696
Name: Delay_Days, dtype: float64
```

The contractor at the **top** has the lowest average delay.

The contractor at the **bottom** has the highest average delay.

---

20. Contractor cost performance

```
contractor_cost = df.groupby("Contractor")[
    "Cost_Variance_Pct"
].mean().sort_values()

print(contractor_cost)
```

```
Contractor
RuralFlow Engineering       1.219687
Sahyadri Water Infra        2.067433
Nirmal Pipeline Works       2.099671
AquaBuild Infra Pvt Ltd     2.274745
Shakti Infra Projects       2.826109
JalSetu Projects            3.398766
Krishna Civil Projects      3.484539
GramJal Contractors         4.351806
Bharat Waterworks           4.612696
BlueLine Water Solutions    5.803426
Name: Cost_Variance_Pct, dtype: float64
```

Interpretation:

Negative → generally under budget

Near 0 → close to budget

Positive → generally over budget

---

21. Contractor delayed-project percentage

```
df["Is_Delayed"] = (
    df["Project_Status"] == "Delayed"
)
```

```
contractor_delay_rate = df.groupby("Contractor")[
    "Is_Delayed"
].mean() * 100
contractor_delay_rate.sort_values(ascending=False)
```

```
Contractor
Shakti Infra Projects       33.333333
BlueLine Water Solutions    32.608696
JalSetu Projects            30.952381
Bharat Waterworks           26.000000
Sahyadri Water Infra        26.000000
GramJal Contractors         24.528302
AquaBuild Infra Pvt Ltd     23.076923
Nirmal Pipeline Works       20.370370
Krishna Civil Projects      19.642857
RuralFlow Engineering       17.307692
Name: Is_Delayed, dtype: float64
```

Now we can say:

Contractor A has a 33% delayed-project rate.

This is more meaningful than saying:

Contractor A has 10 delayed projects.

---

22. Create a contractor performance summary

Let's combine everything:

```
contractor_summary = df.groupby("Contractor").agg(
    Total_Projects=("Project_ID", "count"),
    Average_Delay_Days=("Delay_Days", "mean"),
    Average_Progress=("Actual_Progress_Pct", "mean"),
    Average_Cost_Variance_Pct=("Cost_Variance_Pct", "mean")
)
```

```
delayed_rate = df.groupby("Contractor")[
    "Is_Delayed"
].mean() * 100

contractor_summary["Delayed_Rate_Pct"] = delayed_rate
```

```
contractor_summary
```

Contractor	Total_Projects	Average_Delay_Days	Average_Progress	Average_Cost_Variance_Pct	Delayed_Rate_Pct
AquaBuild Infra Pvt Ltd	52	19.903846	83.944231	2.274745	23.076923
Bharat Waterworks	50	23.340000	82.976000	4.612696	26.000000
BlueLine Water Solutions	46	26.608696	82.915217	5.803426	32.608696
GramJal Contractors	53	18.018868	86.781132	4.351806	24.528302
JalSetu Projects	42	20.642857	87.319048	3.398766	30.952381
Krishna Civil Projects	56	20.642857	84.478571	3.484539	19.642857
Nirmal Pipeline Works	54	25.185185	84.692593	2.099671	20.370370
RuralFlow Engineering	52	21.173077	84.494231	1.219687	17.307692
Sahyadri Water Infra	50	22.740000	86.030000	2.067433	26.000000
Shakti Infra Projects	45	23.888889	79.982222	2.826109	33.333333

---

23. Identify the best-performing contractor

We need to be careful here: "best" depends on what management values.

For example, based primarily on **lowest average delay**:

```
contractor_summary.sort_values(
    "Average_Delay_Days"
).head()
```

Contractor	Total_Projects	Average_Delay_Days	Average_Progress	Average_Cost_Variance_Pct	Delayed_Rate_Pct
GramJal Contractors	53	18.018868	86.781132	4.351806	24.528302
AquaBuild Infra Pvt Ltd	52	19.903846	83.944231	2.274745	23.076923
JalSetu Projects	42	20.642857	87.319048	3.398766	30.952381
Krishna Civil Projects	56	20.642857	84.478571	3.484539	19.642857
RuralFlow Engineering	52	21.173077	84.494231	1.219687	17.307692

Based on **lowest cost variance**:

```
contractor_summary.sort_values(
    "Average_Cost_Variance_Pct"
).head()
```

Contractor	Total_Projects	Average_Delay_Days	Average_Progress	Average_Cost_Variance_Pct	Delayed_Rate_Pct
RuralFlow Engineering	52	21.173077	84.494231	1.219687	17.307692
Sahyadri Water Infra	50	22.740000	86.030000	2.067433	26.000000
Nirmal Pipeline Works	54	25.185185	84.692593	2.099671	20.370370
AquaBuild Infra Pvt Ltd	52	19.903846	83.944231	2.274745	23.076923
Shakti Infra Projects	45	23.888889	79.982222	2.826109	33.333333

There isn't always one definition of "best."

---

24. Look for relationship between cost and delay

Are expensive projects more likely to experience delays?

```
df[
    ["Planned_Cost_INR", "Delay_Days"]
].corr()
```

	Planned_Cost_INR	Delay_Days
Planned_Cost_INR	1.000000	-0.015317
Delay_Days	-0.015317	1.000000

+1 → strong positive relationship

0 → little/no linear relationship

-1 → strong negative relationship

Correlation does not prove causation.

---

25. Look at manpower and delays

Does manpower level appear to be associated with project delays?

```
df[
    ["Total_Manpower", "Delay_Days"]
].corr()
```

	Total_Manpower	Delay_Days
Total_Manpower	1.00000	0.06366
Delay_Days	0.06366	1.00000

Again, this is an association—not proof that manpower causes or prevents delays.

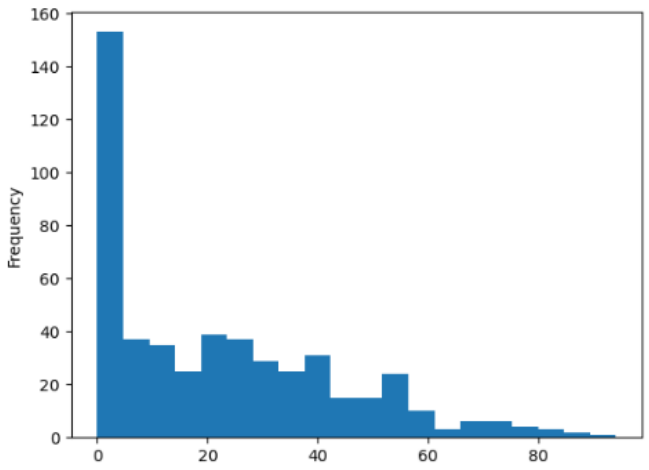
---

26. Create simple visualizations

Delay distribution

```
df["Delay_Days"].plot(
    kind="hist",
    bins=20
)
```

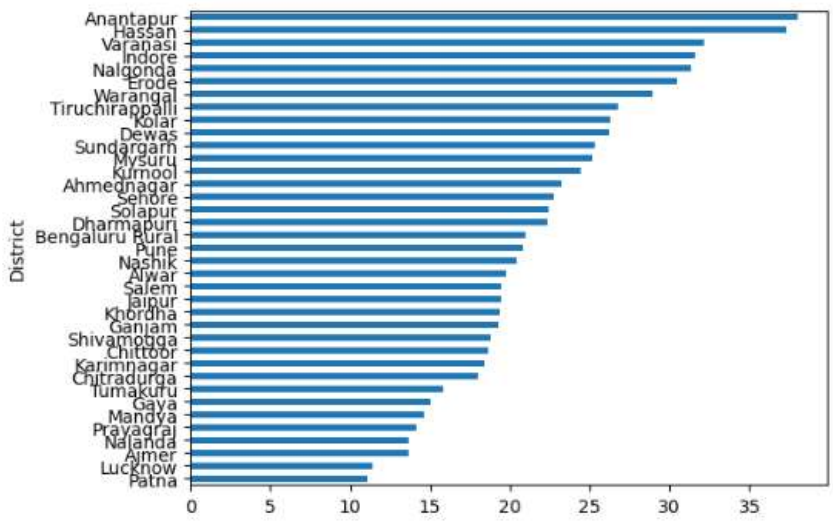
```
<Axes: ylabel='Frequency'>
```



District average delay

```
df.groupby("District")[
    "Delay_Days"
].mean().sort_values().plot(
    kind="barh"
)
```

```
<Axes: ylabel='District'>
```



Planned vs actual cost

```
df[
    ["Planned_Cost_INR", "Actual_Cost_INR"]
].mean().plot(
    kind="bar"
)
```

```
<Axes: >
```

